

Lab. 9
Transactions & Concurrency Control

Content

1. Why some operations must be all-or-nothing
2. What a transaction is: COMMIT and ROLLBACK
3. ACID in one slide
4. Transactions in SQL, then in mysql2
5. The getConnection / beginTransaction / commit / rollback pattern
6. Rolling back on errors and on broken business rules
7. Isolation and the last seat; when to use a transaction
8. Running the starter (npm run dev); hands-on exercises; next: Lab 10
9. Summary

Recap

- Every single SQL statement already succeeds or fails as a whole, one INSERT is all-or-nothing.
- But real operations often take several statements that belong together.
- Enrolling a student means: record the enrolment, and take one seat from the course.
- Those two changes must happen together, or not at all. Today we learn how to guarantee that.

Why Some Operations Must be All-or-nothing?

The Problem: Partial Failure

- Suppose you insert the enrolment, then try to reduce the course's free seats and the second step fails.
- Now there is an enrolment for a seat that was never taken: the data contradicts itself.
- Or the server crashes between the two statements, leaving the change half-done.
- We need a way to say: do all of these, or undo everything and pretend it never started.

What a Transaction is: COMMIT and ROLLBACK

- A transaction groups several statements into one unit of work.
- If every statement succeeds, you COMMIT; the changes become permanent, all at once.
- If anything goes wrong, you ROLLBACK; every change in the group is undone.
- From the outside it is atomic: other users never see a half-finished state.

ACID in One Slide

- Atomicity: all of the statements happen, or none do.
- Consistency: the database moves from one valid state to another, respecting its rules.
- Isolation: concurrent transactions don't trip over each other's half-done work.
- Durability: once committed, the change survives even a crash.

Transactions in SQL, then in mysql2

Setup: a Seats Column; For today's example, give each course a number of free seats. One small change to the schema.

```
ALTER TABLE courses ADD COLUMN seats INT NOT NULL DEFAULT 30;
 UPDATE courses c   SET seats = 10 - (SELECT COUNT(*) FROM
enrolments e WHERE e.course_id = c.id);
-- a course is 'full' when seats = 0
SELECT id, title, seats FROM courses;
```

ALTER TABLE adds the column (you met it in Lab 3 & 4). Enrolling will take one seat; un-enrolling will give it back.

Transactions in SQL, then in mysql2 (Contd.)

You can try a transaction directly in MySQL Workbench before writing any code.

```
START TRANSACTION;

UPDATE courses SET seats = seats - 1 WHERE id = 1;
INSERT INTO enrolments (student_id, course_id) VALUES (3, 1);

COMMIT;     -- make both changes permanent
-- ROLLBACK;  -- ...or undo both, as if nothing happened
```

Run the two statements, then COMMIT to keep them or ROLLBACK to discard them. Try both and watch the data.

Transactions in SQL, then in mysql2 (Contd.)

- A transaction lives on a single connection, all its statements must run on the same one.
- So you cannot use the pool directly; you borrow one connection with pool.getConnection().
- You begin the transaction on that connection, run your statements, then commit or roll back.
- Finally you release the connection back to the pool; always, even if something failed.

Transactions in SQL, then in mysql2 (Contd.)

Lab Exercise 1: *Run a transaction by hand*

- In Workbench, add the seats column to courses with ALTER TABLE.
- Run START TRANSACTION, then reduce a course's seats and insert an enrolment.
- ROLLBACK and confirm both changes vanished; then do it again and COMMIT.
- Note how the data only changes for good after COMMIT.

Starter code:

```
START TRANSACTION;
UPDATE courses SET seats = seats - 1 WHERE id = 1;
INSERT INTO enrolments (student_id, course_id) VALUES (3, 1);
ROLLBACK;  -- then try again with COMMIT;
```

The getConnection / beginTransaction / commit / rollback Pattern

The Pattern: Memorize this shape: borrow, begin, try/commit, catch/rollback, finally/release.

```
const conn = await pool.getConnection();
try {
  await conn.beginTransaction();
  await conn.execute(/* statement 1 */);
  await conn.execute(/* statement 2 */);
  await conn.commit();
} catch (err) {
  await conn.rollback();
  throw err;
} finally {
  conn.release();
}
```

Every transaction you write follows this skeleton. The release in finally returns the connection no matter what.

The getConnection / beginTransaction / commit / rollback Pattern (Contd.)

A Worked Example (Enroll with a Seat): Take a seat and record the enrolment together. The WHERE seats > 0 guards against overselling.

```
const conn = await pool.getConnection();
try {
  await conn.beginTransaction();
  const [u] = await conn.execute(
    'UPDATE courses SET seats = seats - 1 WHERE id = ? AND seats > 0', [courseId]);
  if (u.affectedRows === 0) throw new Error('course is full');
  await conn.execute(
    'INSERT INTO enrolments (student_id, course_id) VALUES (?, ?)', [studentId, courseId]);
  await conn.commit();
} catch (err) { await conn.rollback(); throw err; }
finally { conn.release(); }
```

If no seat was free, affectedRows is 0; we throw, the catch rolls back, and no enrolment is created.

Rolling Back on Errors and on Broken Business Rules

- A rollback isn't only for crashes, you also roll back when a rule is broken.
- 'The course is full' is a rule: there is no error, but the operation must not proceed.
- Throwing inside the try block sends you to the catch, which rolls back the whole group.
- In the controller, turn that into a clear response: 409 Conflict, not a 500.

Rolling Back on Errors and on Broken Business Rules (Contd.)

In a Controller: Wrap the multi-step route in the transaction pattern and map outcomes to status codes.

```
exports.enrol = async (req, res) => {
  const conn = await pool.getConnection();
  try {
    await conn.beginTransaction();
    const [u] = await conn.execute(
      'UPDATE courses SET seats = seats - 1 WHERE id = ? AND seats > 0', [req.body.course_id]);
    if (u.affectedRows === 0) { await conn.rollback(); return res.status(409).json({ error: 'full' }); }
    await conn.execute('INSERT INTO enrolments (student_id, course_id) VALUES (?, ?)',
      [req.params.id, req.body.course_id]);
    await conn.commit(); res.status(201).json({ ok: true });
  } catch (err) { await conn.rollback(); res.status(500).json({ error: err.message }); }
  finally { conn.release(); }
};
```

201 on success, 409 when full, 500 on an unexpected error and the data is consistent in every case.

Running the Starter Project

From the project folder: open a terminal in Lab9_Exercise_Starter (VS Code: Terminal → New Terminal) and run:

npm install	← once: downloads express + mysql2 into node_modules
npm run dev	← starts the server (runs node --watch server.js)
	Lab 9 server on http://localhost:3000
Ctrl + C	← stops the server

"dev" is a script in package.json. *--watch restarts the server every time you save a .js file, so you never need to stop and start it by hand. Open http://localhost:3000 in your browser to use the front-end. Database login: edit user / password in db.js.*

Running the Starter Project (Contd.)

If it does not start, read the first error line:

- **Cannot find module 'express'** → you skipped npm install.
- **Access denied for user 'root'** → set your own user / password in db.js.
- **Unknown database 'lab9'** → run sql/lab9_setup.sql in Workbench first.
- **Unknown column 'seats'** → do Exercise 1 (ALTER TABLE + UPDATE) first.
- **EADDRINUSE :::3000** → the server is already running in another terminal; stop it with Ctrl + C.

Rolling Back on Errors and on Broken Business Rules (Contd.)

Lab Exercise 2: *A transactional enroll*

- Turn your enrol route into a transaction: take a seat, then insert the enrolment.
- Use pool.getConnection() and the begin / commit / rollback / release pattern.
- Guard the seat with WHERE seats > 0 and roll back with 409 when the course is full.
- Test enrolling until the seats run out, and confirm no over-enrolment happens.

Starter code:

```
const conn = await pool.getConnection();
try { await conn.beginTransaction(); /* ... */ await conn.commit(); }
catch (e) { await conn.rollback(); res.status(500).json({ error: e.message }); }
finally { conn.release(); }
```

Isolation and the Last Seat; When to Use a Transaction

- Imagine two students trying to grab the one remaining seat at the very same moment.
- Without care, both could read 'seats = 1' and both enroll, oversold.
- Running each enrolment as a transaction, with the UPDATE ... WHERE seats > 0 as the guard, prevents that.
- The database serializes the two updates: one wins the seat, the other's guard fails and rolls back.

Isolation and the Last Seat; When to Use a Transaction (Contd.)

When to Use a Transaction

- Use one whenever a single logical action spans more than one statement.
- Money: debit one account and credit another, never one without the other.
- Inventory: place an order and reduce stock; cancel and restore it.
- If a half-finished result would be wrong or unsafe, wrap the steps in a transaction.

Isolation and the Last Seat; When to Use a Transaction (Contd.)

Lab Exercise 3: ***Cancel atomically***

- Write an un-enroll route that removes the enrolment and gives the seat back.
- Do both inside one transaction so a seat is never lost or double-counted.
- Return 404 if there was no such enrolment to cancel (and roll back).
- Enroll then cancel, and confirm the course's seats return to where they started.

Starter code:

```
await conn.beginTransaction();
await conn.execute('DELETE FROM enrolments WHERE student_id=? AND course_id=?', [...]);
await conn.execute('UPDATE courses SET seats = seats + 1 WHERE id = ?', [courseId]);
await conn.commit();
```

Good Habits

- Always borrow a single connection for a transaction, and release it in finally.
- Roll back on broken rules as well as on errors; never leave a transaction open.
- Keep transactions short, do the work and commit; don't hold a connection while waiting on other things.
- Let a guarded UPDATE (WHERE seats > 0) do the checking, so two requests can't both win.

Where we're going next

- Your data stays consistent now, even when an action takes several steps.
- As tables grow, the next question is speed: why is a query slow, and how do we fix it?
- In Lab 10 we look at indexing and reading a query plan to make queries fast.

Summary

- A transaction is a group of statements that all commit together or all roll back.
- In SQL: START TRANSACTION ... COMMIT / ROLLBACK. ACID: atomic, consistent, isolated, durable.
- In mysql2: pool.getConnection() then begin / try-commit / catch-rollback / finally-release.
- Roll back on business rules too (course full -> 409); a guarded UPDATE prevents overselling.
- Complete Exercises 1–3 in class, making your enrol and cancel routes atomic.

Thank you & Assignment time…